Laplacian Filtering: Implementation and Performance Analysis

Thanh Nguyen

October 9, 2025

Abstract

This report presents the implementation and performance analysis of parallelizingthe application of Laplacian filters on images. Four parallelization strategies were implemented: sharded rows, sharded columns column major, sharded columns row major, and work queue. Performance was evaluated across varying thread counts, filter sizes, chunk sizes, and image dimensions. Experimental results demonstrate significant speedup potential with proper load balancing, while revealing important trade-offs between synchronization overhead and parallel efficiency. Thread affinity optimization and cache behavior analysis provide insights into achieving optimal performance.

Table of Contents

1 Implementation

1.1 Sequential Baseline

The sequential implementation serves as the performance baseline and correctness reference. For each pixel (i, j) in the output image, the algorithm applies a filter kernel F of size $k \times k$ to the corresponding neighborhood in the input image I. To implement applying a filter kernel to a pixel, a linear mapping must be established from the input image matrix to the filter matrix or vice versa. It is then possible to leverage offsets between the coordinate of the pixel and the dimension of F to obtain the exact bounds of the valid neighborhood in the image I. After convolution, all pixels are normalized using global minimum and maximum values.

1.2 Parallel Implementation Framework

All parallel implementations follow a common two-phase approach:

Phase 1: Parallel Convolution

1. Each thread processes its assigned image region
2. Local minimum and maximum values are tracked
3. Thread-safe updates to global minimum/maximum using mutex locks
4. Barrier synchronization ensures all threads complete before normalization

Phase 2: Parallel Normalization

1. Threads access global minimum/maximum values
2. Each thread normalizes pixels in its assigned region using the same access pattern as the convolution phase

1.3 Sharded Rows

Sharded Rows divides the image into horizontal strips, with each thread responsible for a contiguous set of rows. As this approach accesses data sequentially, it can exploit the benefits of cache's spatial locality.

1.4 Sharded Columns Column Major

This method assigns vertical strips to threads, processing pixels in column-first order. One obvious disadvantage is the reduced cache efficiency due to non-sequential memory access patterns. Therefore, the wider the input image, the higher the cache miss rate which will later be verified via the experiments.

1.5 Sharded Columns Row Major

Sharded Columns Row Major combines column-wise input decomposition with row-major access patterns. Threads are assigned column ranges but process pixels in row-first order within their assigned regions. In terms of cache efficiency, the more columns a thread has in its designated region, the lower the cache miss rate since more data points would be accessed sequentially.

1.6 Work Queue

This strategy decomposes the input image into small, uniform tiles and distribute them to threads through a shared work queue. Work Queue differs fundamentally from the static partitioning approaches seen above as it ensures threads are always busy until every pixel is processed. However, this benefit does come with a cost, which includes synchronization overhead of the shared work queue, cache inefficiency since a thread might not process contiguous tiles, and false sharing when multiple threads process neighboring tiles, causing cache line conflicts.

1.7 Thread Affinity Optimization

To ensure consistent performance measurements and optimal cache utilization, thread affinity was implemented to bind threads to specific CPU cores. Based on the system's hybrid architecture (combining efficiency and performance cores), threads are assigned to physical cores in priority order.

This ensures that threads utilize distinct physical cores rather than competing for resources through hyperthreading, leading to more predictable performance characteristics.

2 Experimental Methodology

2.1 System Configuration

Experiments were conducted on a system with 24 logical cores (16 physical cores with partial hyperthreading). Thread affinity was configured to prioritize physical cores and avoid hyperthreading conflicts. Performance measurements utilized hardware performance counters through the `perf` tool to capture L1 cache behavior and instruction counts.

2.2 General Framework

- Each experiment attempted to isolate the impact of a specific component on the performance of the various processing methods.
- Each experiment was simulated by a Python script.
- The data collected for each configuration was the average of 10 identical measurements to minimize the impact from system overhead
- The key metrics of each participant such as the average run time and the standard deviation were recorded and written to a `txt` file by another Python script.
- Graphs would be produced to provide data visualization.

Note that all 4 experiments can be executed by simply running the Bash script `generate_graphs_script.sh`

2.3 Experiment 1: Impacts of Thread Count on Processing Methods

2.3.1 Design & Setup

This experiment aimed to isolate the impacts of thread count, so the input image and the filter type were fixed throughout. The input image had a width of 6000 and height of 4500 while the filter type was the 5x5 Laplacian filter matrix. Thread count began at 1 and doubled until 16, the

number of physical cores.

For each thread count, the sequential approach and all parallel methods applied the filter matrix to the image for 10 iterations. The run time for each iteration was recorded to compute not only the average runtime so that effects from system overhead could be minimized but also the standard deviation to demonstrate the fluctuation of data. Additionally, speedup was calculated by using the runtime of the sequential algorithm as the benchmark. Lastly, to provide insights into how cache behaviours impacted the performance, L1 cache loads and misses were also recorded.

2.3.2 Experimental Results

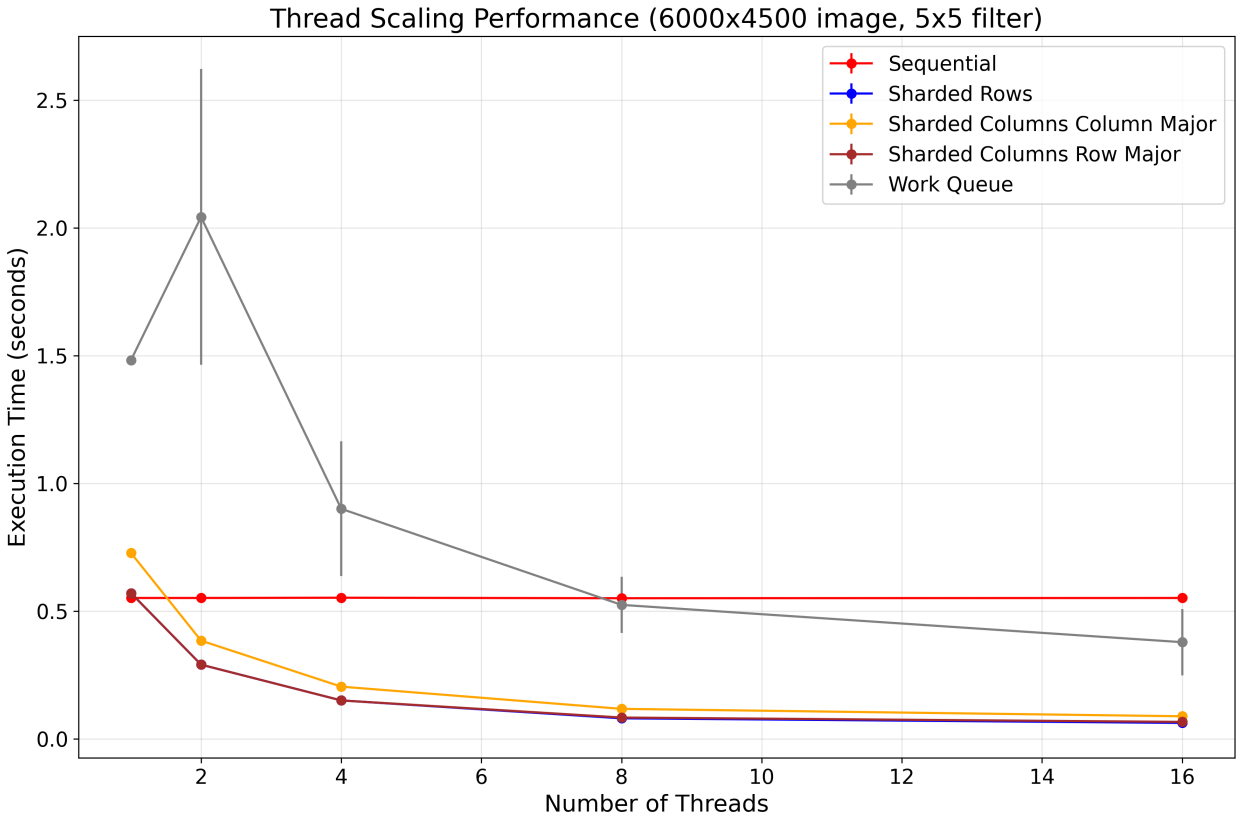

Figure 1: Execution Time Graph

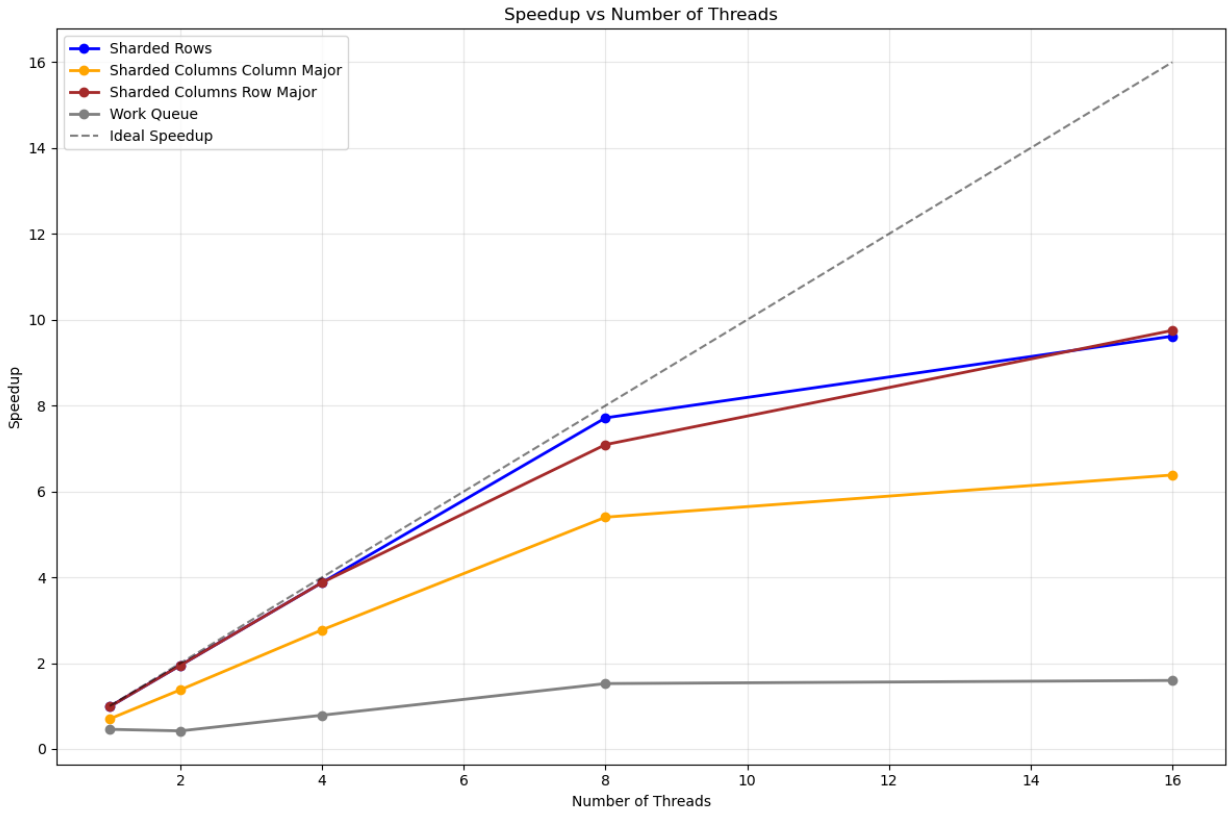

Figure 2: Speedup Graph

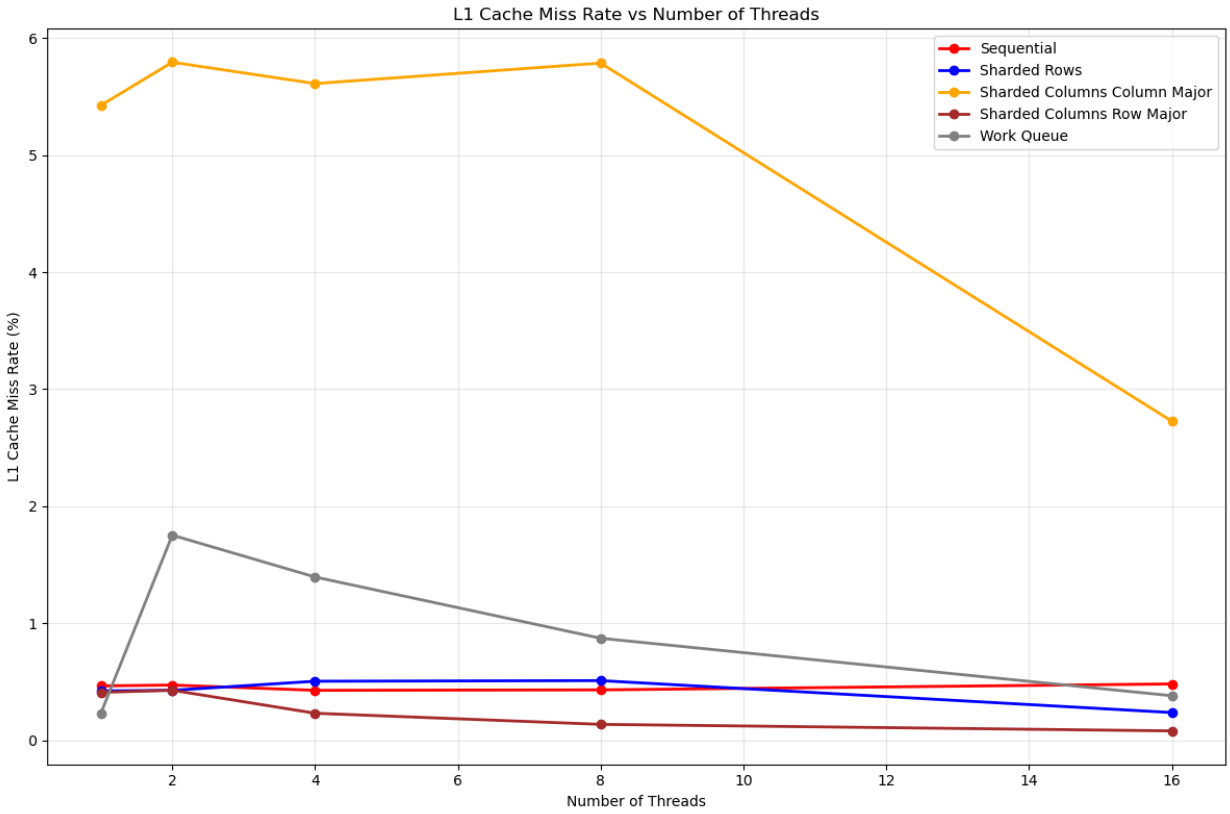

Figure 3: L1 Cache Miss Rate Graph

2.3.3 Performance Analysis

Figure 1 demonstrates the relationship between the number of threads and the execution time of each processing method. Overall, there is a clear trend of diminishing marginal returns where adding more threads do not generate proportionally more improvement in performance in return. In fact, figure 2 serves as a statistical evidence to strengthen this observation as the speedup of all methods detracted further away from the Ideal Speedup line as the thread count increased.

Among all methods, Sharded Rows and Sharded Columns Row Major emerge as the most optimal techniques to decompose data. Their best average runtime were 0.06 and 0.07 seconds respectively. It is not a surprise that these two methods demonstrated such dominating performance in this experiment because of their sequential data access patterns. According to Figure 3, the L1 cache miss rate for Sharded Rows and Sharded Columns Row Major never exceeded 0.5% while other strategies such as Sharded Columns Column Major or Work Queue witnessed at least tripled the miss rate.

Another interesting insight from Figure 1 is that the Work Queue method required the most execution time and caused the most fluctuations in its data points, particularly at a low thread count such as 2. This is likely due to the synchronization overhead and cache misses after context switch.

2.4 Experiment 2: Impacts of Tile Configurations on the Work Queue method

2.4.1 Design & Setup

This experiment focused solely on evaluating the performance of the Work Queue method by varying chunk sizes from 1×1 to 2048×2048, doubling each increment. The input image was fixed to have a width of 2000 and a height of 1500. Moreover, the filter was also fixed to be the 9×9 matrix, so that only the number of threads and the chunk size could vary in the experiment.

2.4.2 Experimental Results

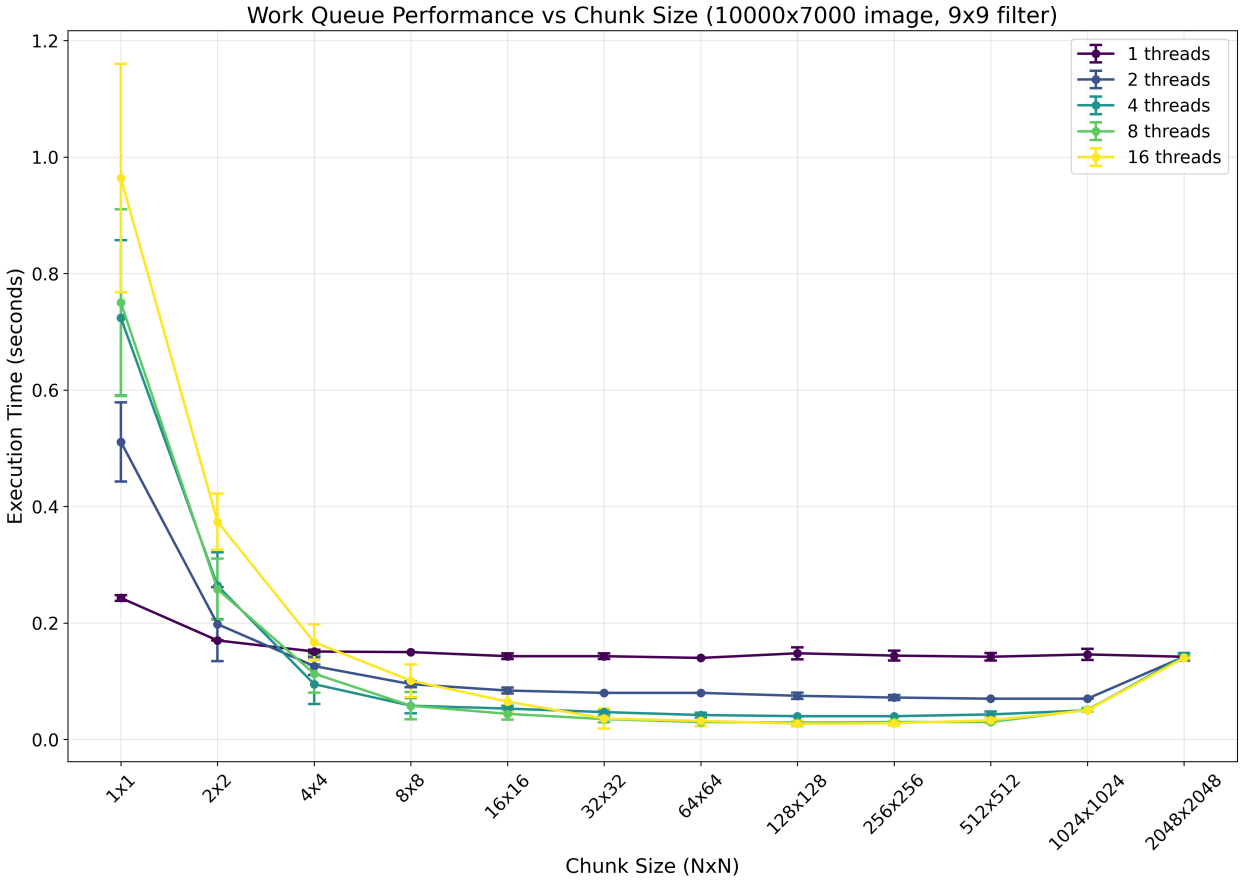

Figure 4: Work Queue Performance Graph

2.4.3 Performance Analysis

Figure 4 illustrates how the choice of chunk size plays a key role when it comes to performance of the Work Queue method. Most notably, 16 threads happened to own both the best and the worst time recorded in the entire graph. Similarly to how section 2.3 explained the cache behaviour of Work Queue, Figure 4 only further amplifies the impact of overhead when number of threads significantly exceeds the chunk size. Not to mention, the error bars attached to each data point also showed how unpreditable the Work Queue method is compared to other static partitioning strategie due to the dynamic tile assignment policy.

2.5 Experiment 3: Impacts of Filter Configurations on Processing Methods

2.5.1 Design & Setup

This experiment seeks to gauge how the configuration of the filter matrix can lead to a difference in performance. The array of possible filters included all the Laplacian filter matrices listed in `filters.cpp`. Each filter was applied to the input image via a processing method with 16 threads.

2.5.2 Experimental Results

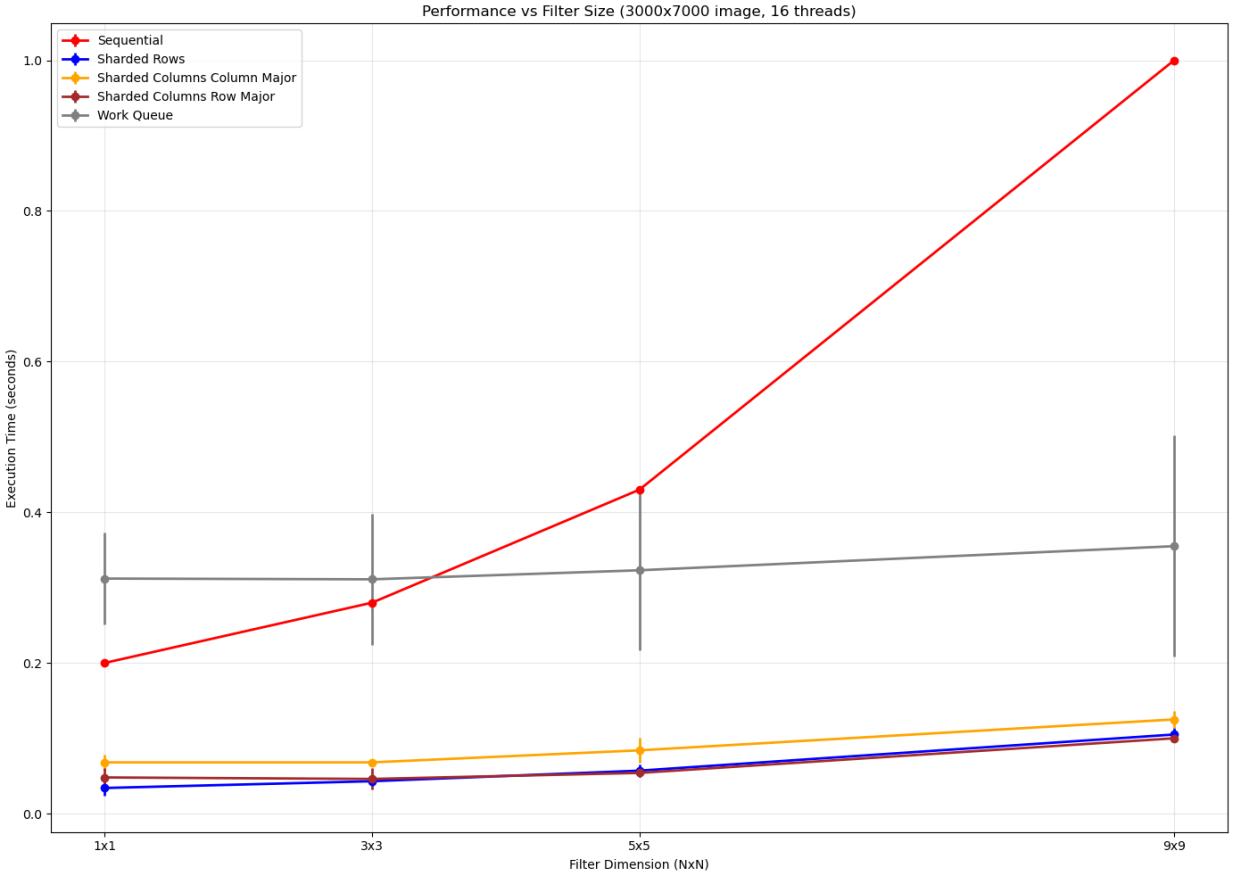

Figure 5: Performance on Various Filters Graph

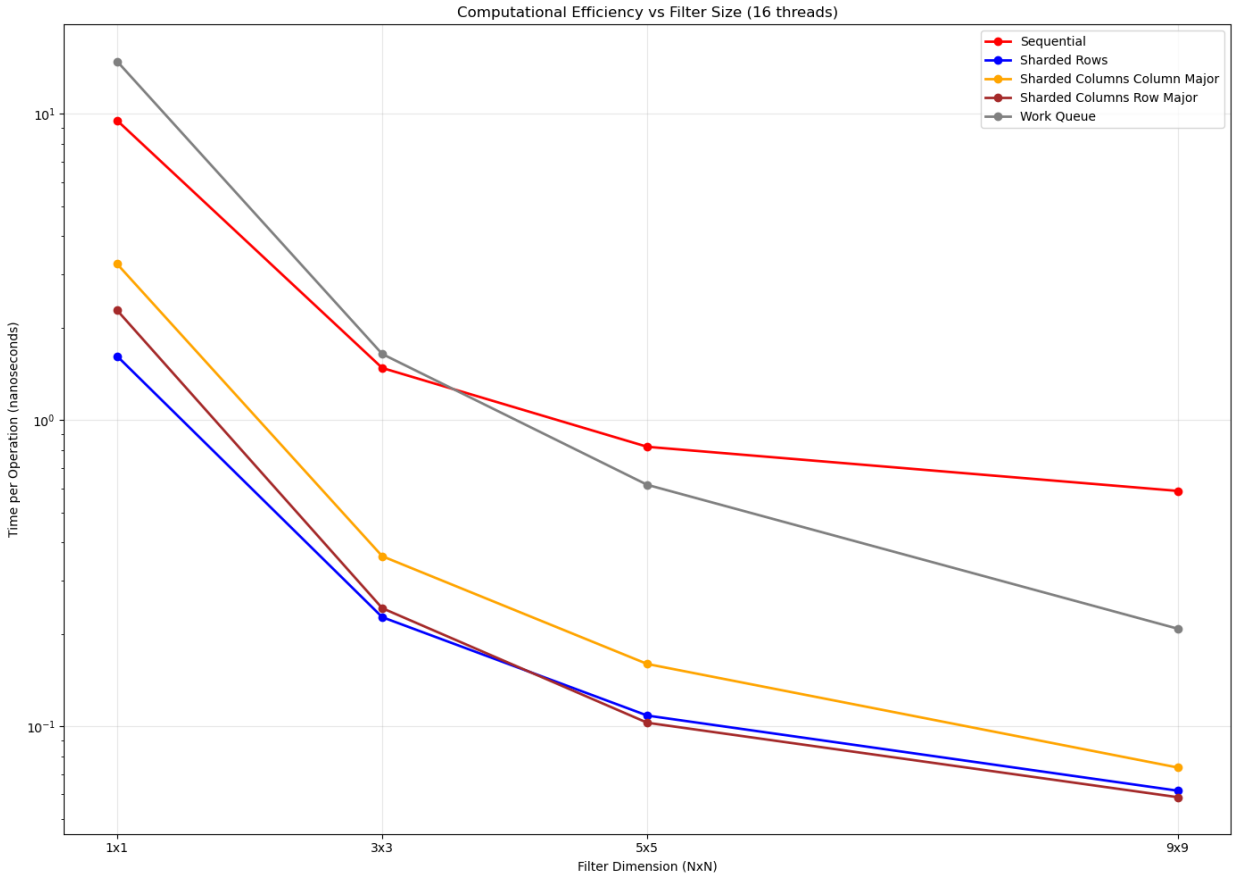

Figure 6: Efficiency on Various Filters Graph

2.5.3 Performance Analysis

From Figure 5, it is no surprise that Sharded Rows and Sharded Columns Row Major once again dominated the competition. The increase in the filter dimension only led to a negligeable increase in the runtime of the two methods. Additionally, it is worth noting that the Work Queue method originally performed worse than the sequential algorithm baseline but as dimension of the filter increases, sequential approach, indeed, becomes the least optimal technique.

Figure 6 plots efficiency against the filter dimension, revealing an unexpected result that greater dimensions of the filter matrix actually reduced the time spent per operation. This is due to the fact that the CPU performs significantly more computations while taking minimal additional time as depicted in Figure 5.

2.6 Experiment 4: Impacts of Image Dimensions on Processing Methods

2.6.1 Design & Setup

This experiment measures how much influence the dimension of an image has on performance. The number of threads was set to 16 while the filter type was fixed to be the 3×3 filter matrix. Each method processed five input images with distinct characteristics.

2.6.2 Experimental Results

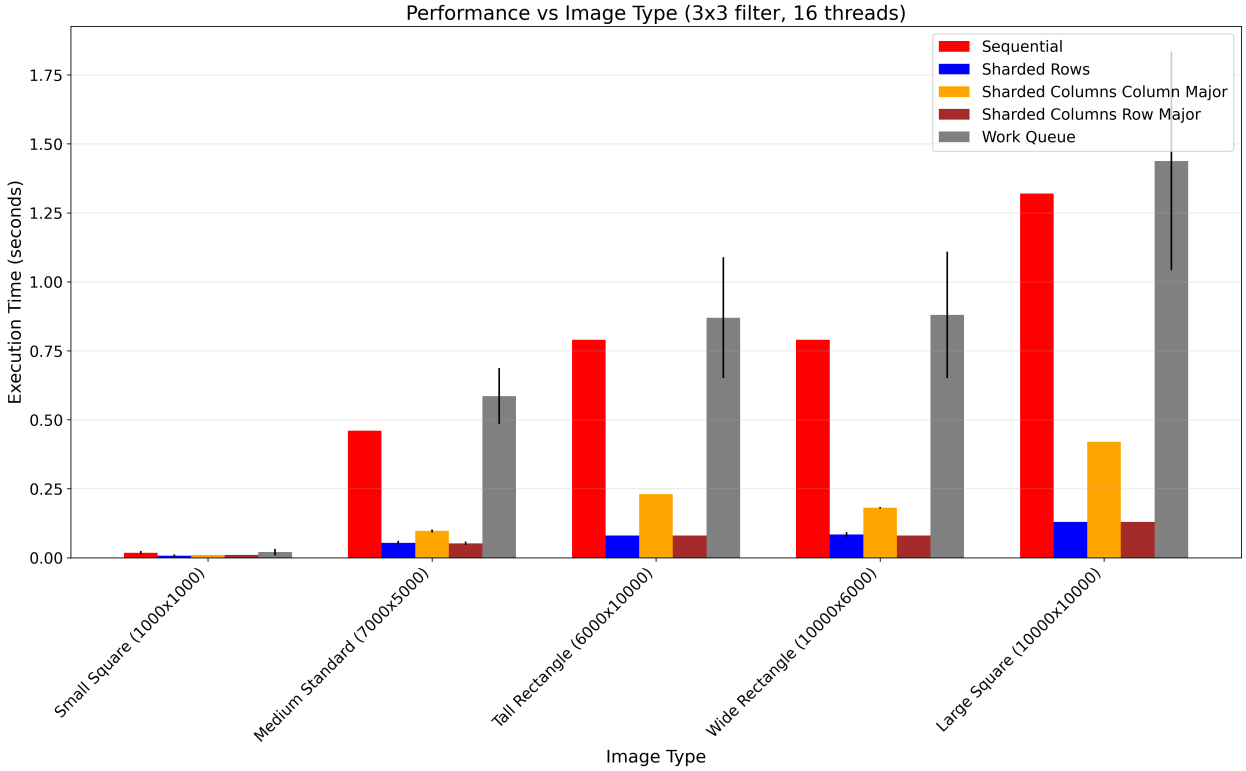

Figure 7: Performance on Various Images Graph

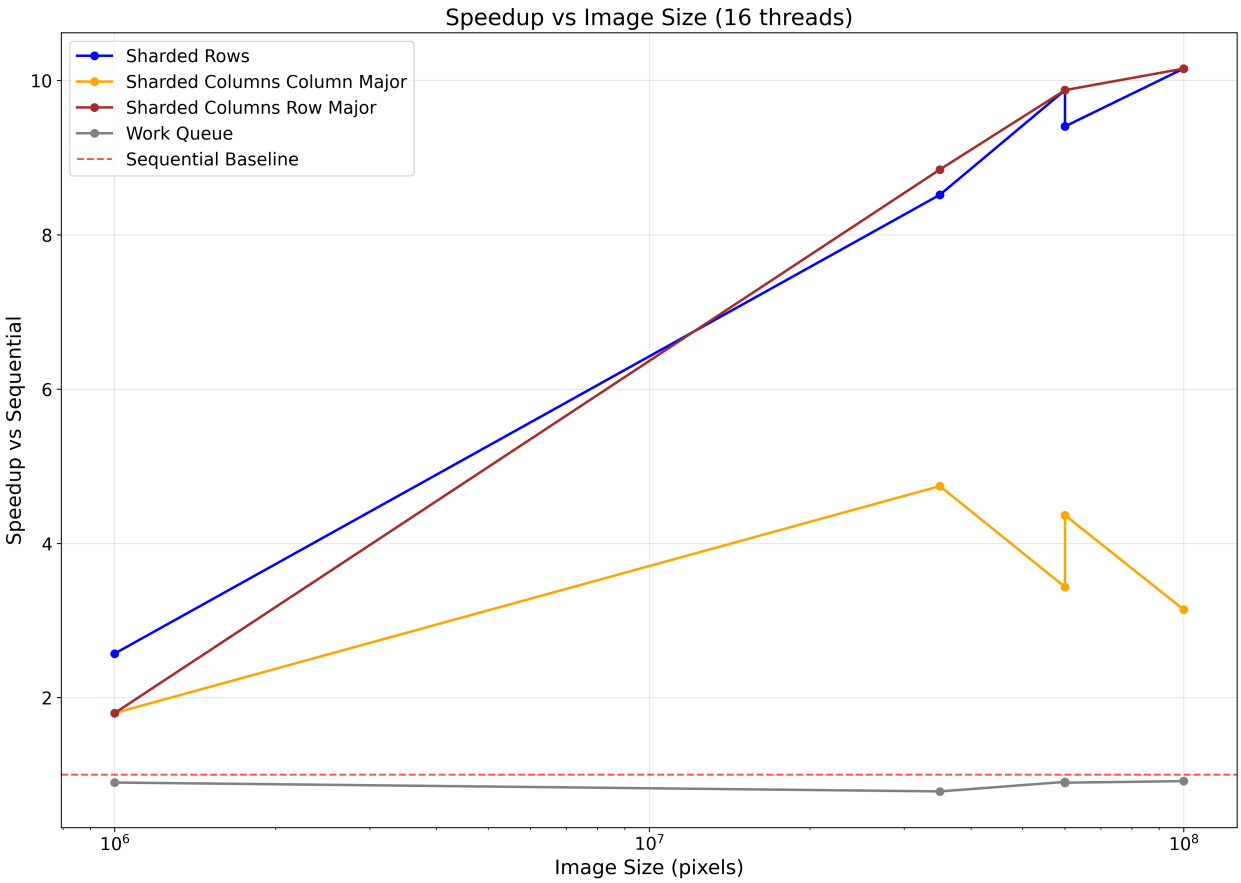

Figure 8: Speedup Graph

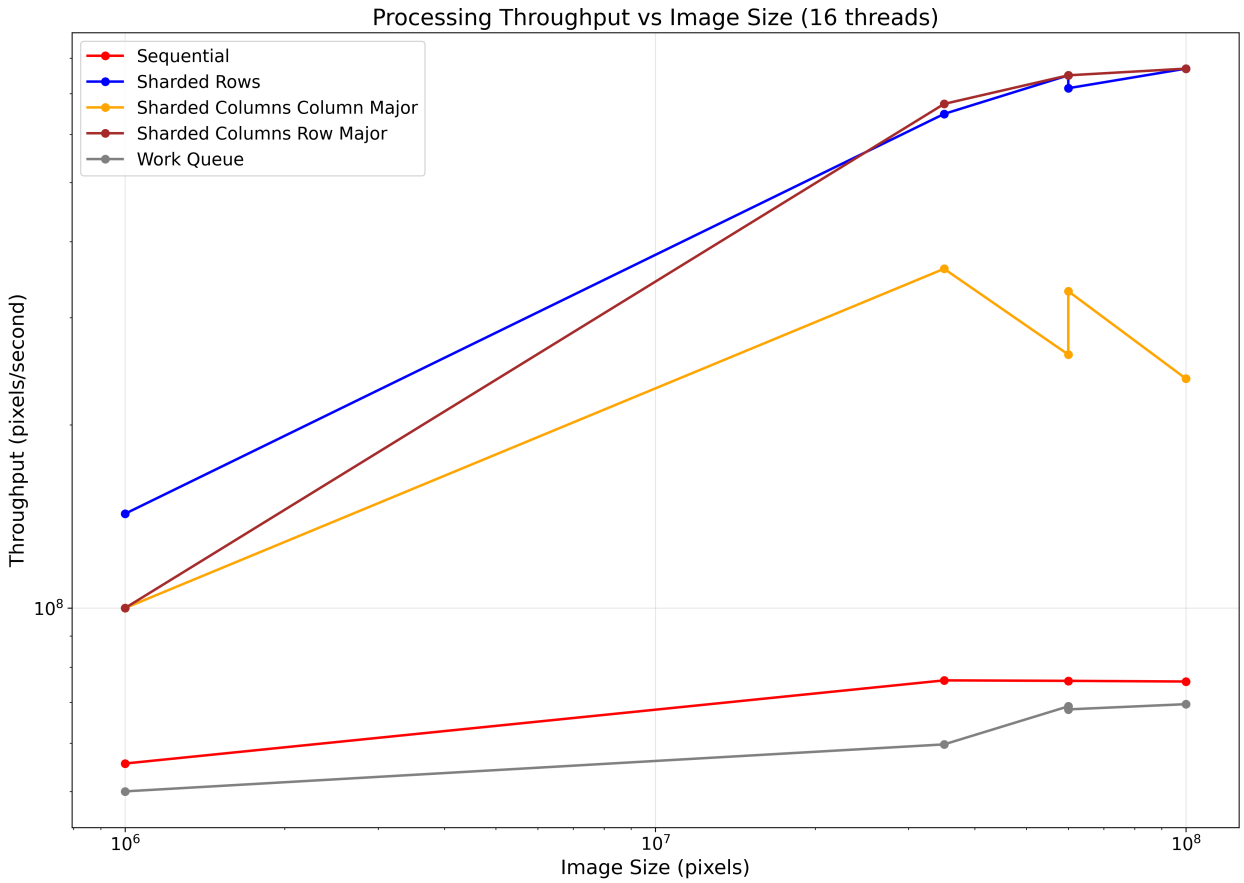

Figure 9: Throughput Graph

2.6.3 Performance Analysis

Figure 7 essentially confirmed that the size of image does not matter in terms of how one method performs relative to another. Once again, the top performers were Sharded Rows and Sharded Columns Row Major, followed by Sharded Columns Column Major with minimal increase in execution time despite extreme changes in the image dimension. Surprisingly, the sequential method was even more efficient than the parallel Work Queue method. This observation illustrate a fundamental principle in parallel computing: when synchronization overhead exceeds the computational savings from parallelization, multi-threading becomes counterproductive rather than beneficial.

3 Conclusion

After collecting and rigorously analyzing the data of the five different processing methods, it is unequivocal that Sharded Rows and Sharded Columns Row Major are the most optimal strategies among the group for the purpose of applying Laplacian filter to an image. Both dominate their counterparts in terms of average runtime and stability to changes in configurations thanks to their cache-friendly sequential data access pattern.

Although the idea of dynamically balancing workload and keeping threads busy of the Work Queue method seems promising theoretically, the overhead cost of synchronization and thread management in practice ultimately outweigh the potential benefits.